```
<p>
<?php $this->lblMessage->Render(); ?><br>
<strong>Title</strong> <?php $this->txtTitle->Render(); ?>
<br>
<strong>Body</strong> <?php $this->txtBody->Render(); ?>
<br>
<?php $this->lblResult->Render(); ?> <br>
<?php $this->btnSubmit->Render(); ?> <br>
</p>
<?php $this->RenderEnd();?>
</body></html>
```

Then visit the *newpost.php* file in your browser; the output should look like Figure 1. Before you think that the *strong* HTML tag at the top is because of an error somewhere, let me tell you that such output is expected; we will know why shortly.

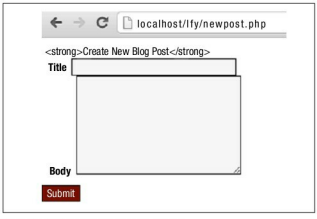

Figure 1: Initial form created by Form_Create()

Understanding the code

QForm separates the presentation layer and program logic into two different files. The *newpost.php* file contains the logic of the page (controller), while the *newpost.tpl.php* governs the layout (view). We begin with the logic.

The first line of the controller file includes the *qcubed.inc.php* file from the root directory of your QCubed installation. This readies the framework capabilities to be used throughout the file. Next, we define a class *NewPost*, which is derived from the QForm class. Before we proceed with creating the HTML controls (such as text-boxes, buttons, etc), we define them as class members. If these variables are not defined as class members but as local variables inside the *Form_Create* function, you would not be able to render them, much less handle any actions on them. This is because the layout is created by the view file, which has no access to variables in the *Form_Create* function but can access class members.

The initial behaviour and content is determined by the *Form_Create* function. In this function, we define the controls (HTML elements). For example, when we do:

```
$this->lblMessage = new QLabel($this);
```

Here we actually tell QCubed to make an *lblMessage* class member variable of type *QLabel*, with the current QForm as the parent. The *$this* variable passed to the *QLabel* constructor indicates that the parent of *lblMessage* is the current QForm class.

The concept of parent is very important to HTML controls in QCubed (QCubed calls them *QControls*). One of the biggest features of the QForm and QControl library is that it allows you to create sophisticated composite controls and panels using other child controls. These controls and panels can have their own layout, defined behaviour and database interactions; they can have their own validation logic and are highly reusable. For example, you could create a calculator panel that can do basic math functions. Once you have created the panel, you can use it on any page by adding just a couple of lines; there is no need to write the same code over and over again for every page. Since controls can be nested, the concept of parent becomes important in controlling the behaviour, placement and visibility.

QLabel is a control that, when rendered, will create text on the page. We set the text to be displayed using the *QLabel*'s *Text* property by doing:

```
$this->lblMessage->Text = "<strong>Create new Blog Post</
strong>";
```

In a very similar fashion, we define other controls. The two text-boxes *txtTitle* and *txtBody* are to allow the user to input the title and body of a blog post; the *btnSubmit* button is to submit these values to the server. For most *QControls*, the *Text* property defines the text they would display. For example, *QLabel*'s *Text* will appear directly where rendered; *QButton*'s *Text* will appear on the button; *QTextBox*'s *Text* will appear as the text in the input box. Every control has some common properties and some unique ones. For example, the *TextMode* property can exist in only *QTextBox*-based controls (*QTextBox*, *QIntegerTextBox* and *QFloatTextbox*) and can convert the text-box to a text-area, a password input control, or an HTML5 'search' text-box.

It is noteworthy that the *Text* of the label *lblResult* was kept blank, only so we could change it to something else later on.

This line:

```
$this->btnSubmit->AddAction(new QClickEvent(), new
QAjaxAction('btnSubmit_Click'));
```

adds an action to the *btnSubmit* button for a *click*